ECE496 Proposal: TinyViT Hardware Acceleration Using a Hybrid CPU–FPGA Architecture on the DE1-SoC

Project ID: 2026272

Supervisor: Professor Vaughn Betz

Administrator: Raymond Tam
Section: 9

Team Members:
Wei-Ni (Winnie) Hsiang
Chenjia Hu
Yiling Zha
Yixin Zha

Date of Submission: October 9, 2026

University of Toronto
Department of Electrical and Computer Engineering

Executive Summary

This project proposes a hardware-accelerated TinyViT image-classification system on the DE1-SoC development platform. The objective is to reduce inference latency relative to execution entirely on the embedded processor while maintaining numerical consistency with a software reference. Although TinyViT is designed as a compact vision transformer, its matrix-multiplication workload remains demanding for embedded computing platforms. Addressing this workload provides an opportunity to investigate how specialized hardware can support transformer inference under practical resource constraints.

The proposed system combines the DE1-SoC's ARM Cortex-A9 processor with its Cyclone V field-programmable gate array (FPGA). The processor will manage image preprocessing, model execution, memory allocation, and operations retained in software, while the FPGA will accelerate selected matrix multiplications. The final prototype will accept an input image and produce a predicted class through an integrated hardware–software inference pipeline.

The accelerator design will investigate a systolic array, in which interconnected processing elements perform parallel multiply-accumulate operations and reuse data as it moves through the array. The proposed arithmetic uses 8-bit integer operands with 32-bit accumulation to reduce operand storage requirements while retaining greater precision for intermediate sums. Because the model exceeds the FPGA's available internal memory, model weights and intermediate activations will reside primarily in processor-side DDR3 memory. Direct memory access transfers and local buffering will supply the accelerator with the data required for each operation.

Preliminary profiling identifies matrix multiplication as a promising acceleration target, with measured processor execution time substantially exceeding the reported data-transfer latency for the tested workload. Evaluation will compare the hybrid implementation against a processor-only baseline using end-to-end inference latency, accelerator execution time, communication overhead, FPGA resource utilization, and numerical consistency. The intended outcome is a functional prototype and a quantitative assessment of the conditions under which selective FPGA acceleration improves TinyViT inference on a resource-constrained embedded platform.

Table of Contents

1. Introduction 4

2. Background and Motivation 4-6

3. Problem Statement 6

4. Project Goal 6

5. Requirements Specification 6-8

6. System Block Diagram 8-9

7. Risk Assessment 9-10

8. Conclusion 10

References 11

Appendix A: Gantt Chart 12

Appendix B: Glossary of Technical Terms and Acronyms 12-14

Appendix C: CPU Baseline 14-15

1.0 Introduction

Vision Transformers have quickly become one of the most popular approaches in computer vision because they deliver top-tier accuracy and use the same AI architecture that powers today's large language models. Deploying vision transformers on embedded platforms requires balancing computational demands against limited processing capacity and memory, and has remained a challenge in the modern technology landscape. Although vision transformers reduce model complexity, matrix multiplications within attention and multilayer perceptron layers remain demanding. A hybrid CPU–FPGA architecture can accelerate these operations through hardware parallelism, but data transfer, buffering, and coordination overhead may limit improvements in overall inference latency.

This project investigates selective TinyViT (a light-weight vision transformer model) acceleration on the DE1-SoC, which integrates an ARM Cortex-A9 Hard Processor System (HPS) with Cyclone V FPGA fabric. The central objective is to determine whether selective acceleration reduces end-to-end inference latency after communication and integration overhead are included. The project will combine parallel accelerator design, reduced-precision arithmetic, memory buffering, and hardware–software integration. Evaluation against an HPS-only baseline will assess video processing frame rate, execution latency, resource utilization, and numerical consistency, establishing the benefits and limitations of the proposed architecture.

2.0 Background and Motivation

This section introduces TinyViT and relevant acceleration research, then explains the platform constraints motivating the proposed hybrid CPU–FPGA design.

2.1 Background

TinyViT is a compact vision-transformer family that uses pretraining distillation to transfer knowledge from larger models. Its 21-million-parameter configuration achieves a published ImageNet-1K top-1 accuracy of 84.8% [3]. Despite its reduced size, attention and multilayer perceptron layers contain demanding matrix multiplications. These regular multiply-accumulate workloads motivate parallel hardware execution on the DE1-SoC, which combines an ARM Cortex-A9 Hard Processor System (HPS) with Cyclone V FPGA fabric. Table A introduces the key technical concepts, while additional terminology is provided in Appendix B.

Table A. Key Technical Terminology

Term	Definition
TinyViT	Tiny Vision Transformer, a compact neural network for image classification.
CPU	Central Processing Unit, a processor that executes software instructions.
FPGA	Field Programmable Gate Array, reconfigurable hardware supporting parallel computation.
HPS	Hard Processor System, the processor subsystem responsible for software execution.
Systolic Array	An architecture using interconnected processing elements for parallel matrix computation.
DMA	Direct Memory Access, a mechanism that transfers data with minimal CPU involvement.
DE1-SoC	The Terasic DE1-SoC is a popular hardware development and education platform built around the Altera Cyclone V System-on-Chip (SoC) FPGA

2.2 Existing Approaches / State of the Art

Existing research addresses transformer efficiency through model compression, quantization, and specialized hardware. Auto-ViT-Acc combines FPGA-aware design with mixed-scheme quantization for vision transformers [1]. I-ViT extends integer-only computation to nonlinear operations such as Softmax, GELU, and LayerNorm, demonstrating that efficient quantization requires attention beyond matrix multiplication [4]. FTRANS combines compressed weight representations with FPGA architecture design for language transformers [5], while FlightLLM emphasizes FPGA-specific computation and memory management [2]. Together, these studies motivate coordinated decisions about precision, parallelism, and data movement. Their different models and platforms mean that their reported gains cannot be assumed for TinyViT on the DE1-SoC. However, few of these approaches emphasize deployment on a cost-effective and resource-limited device like the DE1-SoC, which is what our project is trying to achieve.

2.3 Limitations and Motivation

Preliminary project profiling measured approximately 498 ms for a representative CPU matrix multiplication and 1.4-6.4 ms for associated DMA transfers, suggesting an opportunity for selective offloading. However, approximately 21 MB of raw INT8 weights exceeds the approximately 556 KB of FPGA embedded memory identified in initial resource analysis. Weights and activations must therefore be supplied from HPS DDR3 through local buffering and tiled transfers.

These constraints motivate a hybrid design that retains model control and non-accelerated operations on the HPS while offloading selected matrix multiplications to a systolic-array accelerator using INT8 operands and INT32 accumulation. The central motivation is to determine whether faster arithmetic produces lower inference latency after transfers, synchronization, and conversion costs are included. Evaluation against an HPS-only baseline will measure video processing frame rate and other metrics mentioned in Section 5, establishing the practical benefits and limits of selective FPGA acceleration.

3.0 Problem Statement

TinyViT inference on the DE1-SoC is limited by compute-intensive matrix multiplications on the ARM Cortex-A9 HPS. Preliminary profiling suggests that data transfers are faster than CPU computation, motivating FPGA acceleration. However, its end-to-end benefit remains unproven once communication and coordination overheads are included.

The selected model's approximately 21 MB of INT8 weights exceeds the FPGA's approximately 556 KB of embedded memory, requiring tiled transfers from HPS DDR3. The engineering problem is to partition selected matrix operations between the HPS and FPGA so that computational savings exceed transfer, setup, synchronization, and conversion costs while respecting hardware resource limits. Model control and other operations will remain on the HPS.

The system must perform complete image classification, accelerate at least one selected operation including transfer overhead, and maintain numerical consistency with the software reference. Evaluation will verify INT8 computation with INT32 accumulation and measure end-to-end latency, accelerated-operation latency, communication overhead, and FPGA resource utilization to establish the benefits and limits of selective acceleration.

4.0 Project Goal

The goal of this project is to develop an end-to-end TinyViT video classification pipeline on the DE1-SoC, targeting sustained processing of at least 1 frame per second (FPS) while maintaining classification accuracy.

5.0 System Requirements Specification

The project specifications are divided into requirements, constraints, and objectives, defining the system's essential functionality, platform limitations, and performance targets.

5.1 Requirements

The system must perform end-to-end TinyViT inference, accelerate at least one matrix operation by 50%

(including transfer overhead), and maintain 95% prediction agreement with the CPU baseline. Computation and communication latency must be measured separately.

Table B: System Requirements

ID	Specification	Description
R1	Complete TinyViT image classification.	Produce a valid predicted class for 100% of test images.
R2	Accelerate at least one MSA or MLP matrix operation.	≥50% latency reduction vs. HPS-only, including transfer overhead.
R3	Maintain classification consistency.	≥95% top-1 prediction agreement with CPU reference.
R4	Measure computation and communication latency.	Report end-to-end, kernel, and transfer latency in ms, averaged over ≥100 runs.

5.2 Constraints

The implementation must use the DE1-SoC's ARM Cortex-A9 HPS and FPGA, operating primarily within its available logic, DSP, and memory resources. External resources are permitted only when necessary.

Table C: System Constraints

ID	Specification	Description
C1	Use the DE1-SoC development board.	Dual-core ARM Cortex-A9 and Cyclone V FPGA.
C2	Operate within available FPGA resources.	≤85K logic elements, ≤87 DSP blocks, ≤4.45 Mb FPGA block memory.
C3	Use on-board memory where possible.	1 GB HPS DDR3 and 64 MB FPGA SDRAM.
C4	Execute the selected TinyViT workload.	Fixed TinyViT model and input resolution for CPU–FPGA comparison.

5.3 Objectives

The design aims to improve video processing throughput to 1 FPS compared to the CPU baseline, minimize communication overhead, and improve FPGA resource efficiency without sacrificing classification consistency. A preliminary CPU baseline provides timings and verified outputs for selected INT8 matrix operations, and full-model HPS latency for one inference is 5.7s. Preliminary CPU performance measurements are provided in Appendix C.

Table D: System Objectives

ID	Specification	Description
O1	Minimize HPS–FPGA transfer overhead.	≤20% of accelerated-operation latency.
O2	Optimize FPGA resource utilization	Target ≤90% utilization of logic, DSP, and block memory.
O3	Maximize video processing throughput.	Target ≥1 FPS sustained over a continuous 1-min test, representing a ≥5.7× throughput improvement over the 5.7-second CPU inference baseline.

6.0 System Block Diagram

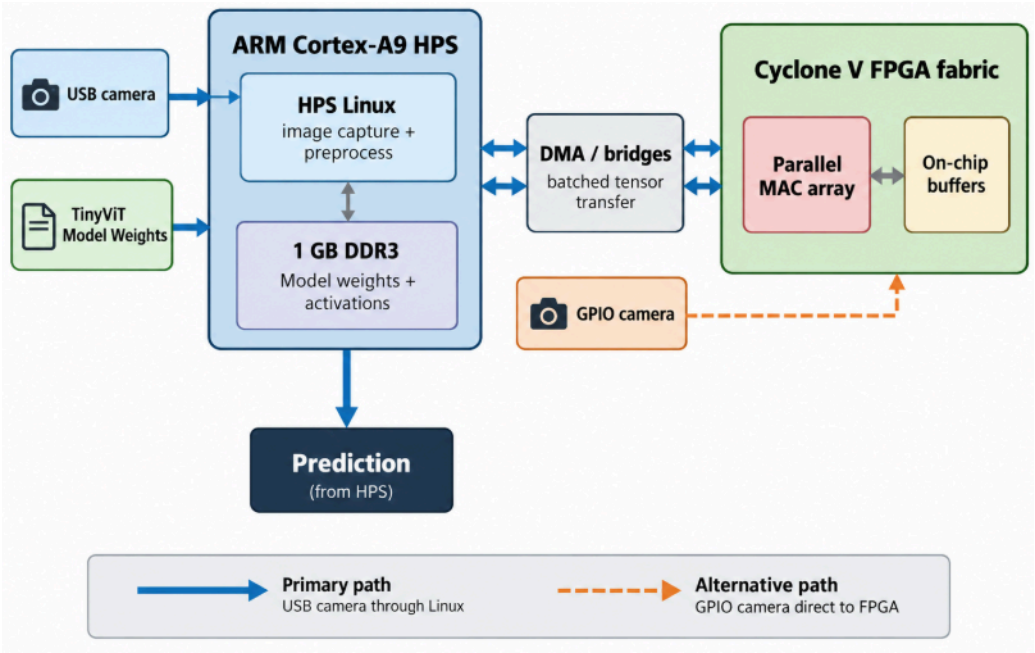


Figure 1. System block diagram

The system block diagram in Figure 1 illustrates the overall inference workflow. Before any processing begins, the TinyViT model weights are loaded into the HPS DDR3 memory. The USB camera serves as the input module, feeding a continuous video stream to the DE1-SoC board. The HPS program will process the input video stream and extract individual frames, which will then be fed into our software pipeline for image classification. Once the software pipeline encounters an operation selected for acceleration, it will pause execution and offload the operation to our FPGA fabric, which contains an acceleration tile. Model weights will be transferred via DMA into the on-chip Random Access Memory (RAM) buffers. Once hardware acceleration is complete, the software will read back the results and

continue with the rest of the pipeline. Finally, after the entire software pipeline finishes, the predicted class label will be output.

7.0 Risk Assessment

The main risks include FPGA resource limitations, HPS–FPGA communication overhead, reduced numerical precision, and integration delays. These may limit acceleration, affect output consistency, or delay system completion. In particular, transfer overhead may offset improvements in kernel execution time [1], [2].

Mitigation will focus on early resource validation, independent subsystem testing, and incremental integration. Regular synthesis will identify resource limitations, while separate kernel and transfer latency measurements will assess system-level performance.

Table E. Risk Assessment and Mitigation Strategies

ID	Risk	Likelihood	Consequence	Mitigation
RK1	Accelerator or buffering may exceed DE1-SoC resources.	Medium–High	The design may not fit or may require reduced parallelism.	Reduce parallelism or buffer size, use tiling, and use external memory if necessary.
RK2	HPS–FPGA transfer overhead may reduce end-to-end speedup.	Medium	FPGA acceleration may provide limited overall latency improvement.	Batch transfers, reuse data, and prioritize high arithmetic-intensity operations.
RK3	Reduced precision may change inference outputs.	Medium	Accelerated outputs may differ from the CPU reference.	Compare against the CPU baseline and increase precision where required.
RK4	Integration and debugging may take longer than planned.	Medium	Full end-to-end integration may not be completed on schedule.	Test subsystems independently and integrate incrementally.

FPGA resource constraints and communication overhead require balancing parallelism, buffering, and data movement to maintain efficient acceleration [1], [2]. Quantization can improve throughput but may reduce inference accuracy, requiring validation against the CPU baseline [1].

If full integration is delayed, the CPU baseline with peripheral support will serve as a fallback for the complete image classification pipeline. This approach allows evaluation of kernel latency, transfer

overhead, resource utilization, and output consistency without requiring full integration of the FPGA-accelerated inference pipeline.

8.0 Conclusion

This project proposes a hybrid CPU–FPGA system for TinyViT classification of video frames on the DE1-SoC. Selected matrix multiplications will execute on the Cyclone V FPGA, while the ARM Cortex-A9 HPS manages preprocessing, model control, and remaining operations. The proposed systolic-array accelerator uses INT8 operands, INT32 accumulation, DMA transfers, and tiled buffering.

The expected contributions are an integrated inference prototype and a quantitative evaluation of selective acceleration. Targets include at least 50% latency reduction for one selected matrix operation, including transfer overhead, at least 95% top-1 prediction agreement with the CPU reference, and a target throughput of 1 FPS. The broader goal of sustained processing at 1 FPS will be assessed through full-system benchmarking.

An approximately eight-month schedule, prior DE1-SoC experience, and incremental verification support the project's feasibility. Measurements of latency, communication overhead, and resource utilization will establish how hardware–software partitioning affects performance under embedded resource constraints, informing accelerator design for visual monitoring and automated inspection.

References

[1] Z. Li et al., “Auto-ViT-Acc: An FPGA-aware automatic acceleration framework for vision transformer with mixed-scheme quantization,” in Proc. 32nd Int. Conf. Field-Programmable Logic and Applications (FPL), Belfast, U.K., 2022, pp. 109–116, doi: 10.1109/FPL57034.2022.00027.

[2] S. Zeng et al., “FlightLLM: Efficient large language model inference with a complete mapping flow on FPGAs,” in Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA ‘24), Monterey, CA, USA, 2024, pp. 223–234, doi: 10.1145/3626202.3637562.

[3] K. Wu et al., “TinyViT: Fast pretraining distillation for small vision transformers,” in Proc. Eur. Conf. Comput. Vis. (ECCV), Tel Aviv, Israel, 2022, pp. 68–85, doi: 10.1007/978-3-031-19803-8_5.

[4] Z. Li and Q. Gu, “I-ViT: Integer-only quantization for efficient vision transformer inference,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), Paris, France, 2023, pp. 17065–17075.

[5] B. Li et al., “FTRANS: Energy-efficient acceleration of transformers using FPGA,” in Proc. ACM/IEEE Int. Symp. Low Power Electron. Design (ISLPED), Boston, MA, USA, 2020, pp. 175–180, doi: 10.1145/3370748.3406567.

Appendix A: Gantt Chart

Project Gantt Chart

Project 2026-09-01

Today 2026-10-09

Yellow cells = edit these. Grey cells are formulas (End, Status) — leave them.

Bars draw automatically from Start + Duration; the darker part shows % complete. Red column = this week.

ID	Task / Phase	Owner	Start	Duration (days)	End	% Done	Status
1	**Hardware design & verification**						
1.1	Systolic array design & verification	Chenjia	2026-09-01	30	2026-09-30	100%	Done
1.2	Systolic array optimization	Chenjia	2026-10-01	11	2026-10-11	90%	In progress
1.3	Activation function accelerator design & verification	Winnie	2026-10-01	30	2026-10-30	20%	In progress
1.4	Peripheral setup	Tiffany	2026-10-14	40	2026-11-22	0%	Not started
1.5	Hardware unit integration	Chenjia & Winnie & Tiffany	2026-11-01	50	2026-12-20	0%	Not started
2	**Software integration**						
2.1	Modify MNN backend	Stephanie	2026-10-01	60	2026-11-29	10%	In progress
2.2	Software accuracy and performance test	Stephanie	2026-11-29	14	2026-12-12	0%	Not started
3	**Baseline & quantization measurement**						
3.1	Run baseline accuracy and performance test	Tiffany	2026-10-01	14	2026-10-14	10%	In progress
3.2	Compare different quantization scheme	Tiffany	2026-10-01	14	2026-10-14	10%	In progress
4	**Integration**						
4.1	Software & hardware integration	Team	2026-12-21	30	2027-01-19	0%	Not started
4.2	System optimization (software down to hardware)	Team	2027-01-20	45	2027-03-05	0%	Not started



Figure 2. Gantt Chart

Appendix B: Glossary of Technical Terms and Acronyms

This appendix provides additional definitions of the technical terminology used throughout the proposal.

Table F. Additional Technical Terminology

Term	**Definition**
AI	Artificial Intelligence, computer systems capable of performing tasks that typically require human intelligence.
SoC	System on Chip, an integrated circuit combining multiple computing components.
DE1 SoC	A development board integrating an ARM Cortex A9 processor system and Cyclone V FPGA.
ARM	A processor architecture commonly used in embedded systems.
ViT	Vision Transformer, a neural network architecture that applies Transformer techniques to image processing.
MSA	Multihead Self Attention, a mechanism that captures relationships between input features.

MLP	Multilayer Perceptron, a neural network component consisting of fully connected layers.
MAC	Multiply Accumulate, an arithmetic operation that multiplies values and accumulates the results.
PE	Processing Element, an individual computational unit within a hardware accelerator.
DDR3	Double Data Rate 3, a type of synchronous dynamic memory used by the HPS.
SDRAM	Synchronous Dynamic Random Access Memory, memory synchronized with a clock signal.
RAM	Random Access Memory, memory used for temporary data storage.
LUT	Look Up Table, a configurable FPGA resource used to implement digital logic.
DSP	Digital Signal Processing, specialized FPGA hardware resources for arithmetic operations.
INT8	8 bit integer representation used to reduce computational and storage requirements.
INT32	32 bit integer representation used for higher precision accumulation.
Quantization	Reducing numerical precision to decrease computational and memory requirements.
Tiling	Dividing large computations into smaller blocks to improve memory efficiency.
Buffering	Temporarily storing data to reduce repeated memory transfers.
Inference	Using a trained machine learning model to generate predictions.
Latency	The time required to complete a computation or inference task.
Throughput	The number of operations or inference tasks completed per unit of time.
Offloading	Transferring computational tasks from a processor to specialized hardware.
Top 1 Accuracy	The percentage of predictions where the highest ranked class matches the correct answer.

ImageNet 1K	An image classification dataset containing 1,000 object categories.
Softmax	A function that converts numerical scores into a normalized probability distribution.
GELU	Gaussian Error Linear Unit, an activation function used in neural networks.
LayerNorm	Layer Normalization, a technique that normalizes neural network activations.
FPGA Fabric	The programmable logic, memory, arithmetic, and routing resources within an FPGA.
Numerical Consistency	The degree of agreement between accelerated results and a reference implementation.
ms	Millisecond, one thousandth of a second.
KB / MB	Kilobyte and Megabyte, units of digital storage capacity.
Mb	Megabit, a unit of digital storage capacity measured in bits.
FPS	Frames per second, the number of image frames processed by the complete inference pipeline per second.

Appendix C: CPU Baseline

This appendix provides detailed information of CPU matrix measurements obtained through single-threaded C code running on DE1-SoC HPS. Inputs are synthetic signed INT8 values. Each larger case used three warmups and 20 individually timed executions per arithmetic rule.

Table G. CPU baseline

Operation	**K, N, M in W[K,N] × X[N,M]**	**Full-accumulation median ms**	**RTL-compatible median ms**	**RTL-compatible p95 ms**
QKV, one window	576, 192, 49	22.768	53.479	53.643
MLP FC1	768, 192, 784	619.096	1,130.260	1,132.105
MLP FC2	192, 768, 784	971.128	1,137.588	1,140.215

Full accumulation converts once after the complete dot product. RTL compatibility converts 16-term partial sums and merges them with saturation. Both columns measure CPU execution. All 22 case/rule combinations passed their own golden-output checks; this does not establish equality between the two rules or correctness of an FPGA implementation. The reported p95 is preliminary with 20 samples. QKV represents one window, whereas MLP uses a larger token scope; the three times must not be summed and labelled full-model latency.

For the proposal's minimum 50% latency reduction, provisional maximum offload medians against the current RTL-compatible CPU routine are **26.740 ms** for QKV, **565.130 ms** for FC1 and **568.794 ms** for FC2. For the **3×** stretch objective they are **17.826 ms, 376.753 ms and 379.196 ms** respectively. All budgets refer to the complete corresponding operation including overhead, and must be reassessed against an optimized CPU comparator.